



# Simple but not simpler: A surface-sliding method for finding the minimum distance between two ellipsoids

Dariusz Amirkhani, Junfeng Zhang<sup>ID</sup>\*

School of Engineering and Computer Science, Laurentian University, 935 Ramsey Lake Road, Sudbury, Ontario, P3E 2C6, Canada

## ARTICLE INFO

### Keywords:

Ellipsoid distance  
Surface-sliding method  
Geometric algorithms  
Collision/contact detection  
Particle-resolved simulation  
Computer graphics

## ABSTRACT

We propose a novel iterative process to establish the minimum separation between two ellipsoids. The method maintains one point on each surface and updates their locations in the  $\theta - \phi$  parametric space. The *tension* along the connecting segment between the two surface points serves as the guidance for the sliding direction and the distance between them thus decreases gradually. The minimum distance is established when the connecting segment becomes perpendicular to the ellipsoid surfaces, whereas the net effect of the segment tension disappears and the surface points are not moving anymore. Demonstration examples and benchmark validations are thoroughly conducted, and satisfactory numerical performance is observed, including accuracy, consistency, stability, and robustness. Furthermore, compared to other existing techniques, this surface-sliding approach has several attractive features, such as the clear geometric representation, the concise formulation, the simple algorithm and the potential to be extended to other situations straightforwardly. This method is expected to be useful for future studies in computer graphics, engineering design, material modeling, and scientific simulations.

## 1. Introduction

Finding the minimum distance between two ellipsoids in the three-dimensional (3D) space is frequently encountered in various engineering and research computations, such as collision and contact detection in robotics, particle-resolved simulations of suspension flows, material modeling and computer graphics [1–7]. In such applications, one needs not only the scalar separation distance but also the associated closest points and contact direction for collision forces or near-field lubrication interactions [1,8–10].

A variety of algorithms have been proposed for this geometric query, and these methods have been recently reviewed by Girault et al. [1] and Banna [11]. A widely used geometric approach is the moving-ball algorithm, which iteratively constructs interior tangent spheres and updates surface points as the intersection points of the line segment connecting the sphere centers with ellipsoid surfaces [12]. The appeal lies in its conceptual simplicity; however, slow convergence has been reported for small, thin ellipsoids [1,12]. On the other hand, convex-geometry approaches, instead, reduce the distance computation to a distance-to-origin problem for the Minkowski difference of two convex sets, leading to procedures such as the Gilbert–Johnson–Keerthi algorithm [1,13,14]. These methods require support mappings and a simplex-update sub-algorithm, and recovering the closest points on the original ellipsoids involves an additional reconstruction step [1].

Other formulations include the physics-inspired dynamics approaches such as the Newton–Coulomb (charged-body) method [15] and the optimization-based methods such as the exact exterior penalty formulations [16]. Comparative tests indicate that these methods can introduce nontrivial parameter calibration, overlap-prechecks, or substantial computational overhead, and they may be less attractive for large numbers of repeated distance queries in simulations [1]. Furthermore, algebraic approaches based on Lagrange multipliers and elimination theory reduce the constrained minimum-distance problem to solving high-degree polynomial systems [17]. While mathematically rigorous, comparative discussions emphasize that such implementations can be laborious and challenging to deploy robustly in practice [1, 16], and solving high-degree polynomial root problems can be numerically delicate in floating-point arithmetic [18]. More recently, a machine learning approach has also been developed for estimating the distance between ellipsoids [11].

Regardless of the mathematical formulations and computational routes, the minimum-distance configuration satisfies a simple geometric characterization: when a pair of closest points exists, the segment joining the closest points aligns with the outward normals at both surface points [1,12]. Inspired by the moving-ball method [12], in this work, we propose an alternative to find the minimum distance and the corresponding locations on two ellipsoids by allowing the two

\* Corresponding author.

E-mail address: [jzhang@laurentian.ca](mailto:jzhang@laurentian.ca) (J. Zhang).

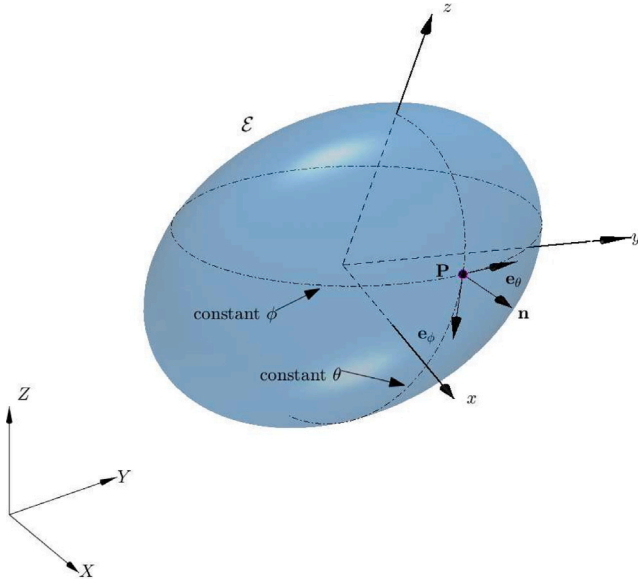


Fig. 1. A graphic representation for the notations adopted in this article for an ellipsoid.

points, one on each ellipsoid, to slide smoothly on the surfaces under the direction of the connecting segment. Compared to other existing methods in the literature, no high-degree polynomial solver is needed, and the segment-surface intersection calculations in the moving-ball method are avoided. The algorithm is simple, straightforward and easy to implement, and explicit information is readily available during the search iteration process without reconstruction. Numerical tests demonstrate the calculation is efficient, accurate and stable. These features are valuable in situations, such as particle-resolved simulations, where frequent and extensive calculations are needed for ellipsoid clusters.

The rest of this paper is organized as follows. In Section 2, we first present the relevant mathematical formulations, and then describe the surface-sliding iteration step in detail with an algorithm pseudo-code. The numerical complexity and memory footprint are also discussed. Several numerical examples are provided in Section 3 to demonstrate the algorithm performance in accuracy, robustness and spatial isotropy. Benchmark validations against analytical solution are also carried out and the qualitative comparison to other minimum distance methods are presented. Finally, summary and concluding remarks are presented in Section 4.

## 2. Method description

### 2.1. Coordinate systems and ellipsoid representation

We first consider one ellipsoid  $\mathcal{E}$  in the global coordinate  $\mathbf{X} = [X, Y, Z]^T$  in Fig. 1. A local coordinate system  $\mathbf{x} = [x, y, z]^T$  is also established at the ellipsoid center  $\mathbf{X}_0$  and along its axes. The semi-axis lengths are denoted by  $a$ ,  $b$  and  $c$ , respectively, along the local coordinate directions  $x$ ,  $y$  and  $z$ . The transformation between the global and local coordinates for a position can be expressed by

$$\mathbf{X} = \mathbf{R}\mathbf{x} + \mathbf{X}_0, \quad (1)$$

where  $\mathbf{R}$  is the rotation matrix given as

$$\mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix} \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (2)$$

Here  $\alpha$ ,  $\beta$  and  $\gamma$  are the Euler angles, which describe the relative orientation between the global and local coordinate directions [19].

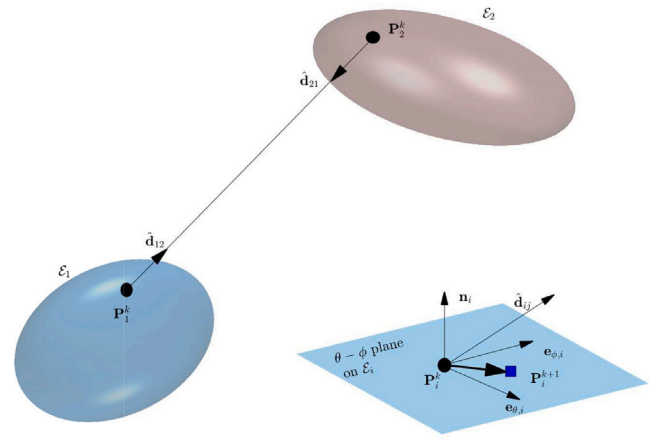


Fig. 2. A graphic representation of the search process for the minimum distance between two ellipsoids, with key vectors used in the calculation illustrated.

The local coordinates of a point  $\mathbf{P}$  on the ellipsoid surface  $\mathcal{E}$  can be conveniently expressed using the parametric representation

$$\mathbf{x} = [a \sin \phi \cos \theta, b \sin \phi \sin \theta, c \cos \phi]^T, \quad (3)$$

where  $\phi \in [0, \pi]$  is the polar angle measured from the  $z$  axis, and the parameter  $\theta$  varies between 0 and  $2\pi$ . Therefore, any location on the surface  $\mathcal{E}$  is uniquely defined by a pair of these two parameters  $(\theta, \phi)$ . Furthermore, normal ( $\mathbf{n}$ ) and tangent ( $\mathbf{e}_\theta$  and  $\mathbf{e}_\phi$ ) directions at point  $\mathbf{P}$  can also be obtained from them as

$$\mathbf{n} = \frac{\mathbf{N}}{|\mathbf{N}|}, \quad \mathbf{N} = [bc \sin \phi \cos \theta, ac \sin \phi \sin \theta, ab \cos \phi]^T; \quad (4)$$

$$\mathbf{e}_\theta = \frac{\mathbf{r}_\theta}{|\mathbf{r}_\theta|}, \quad \mathbf{r}_\theta = [-a \sin \phi \sin \theta, b \sin \phi \cos \theta, 0]^T; \quad (5)$$

$$\mathbf{e}_\phi = \frac{\mathbf{r}_\phi}{|\mathbf{r}_\phi|}, \quad \mathbf{r}_\phi = [a \cos \phi \cos \theta, b \cos \phi \sin \theta, -c \sin \phi]^T. \quad (6)$$

All these unit vectors are given here in the local coordinates. The conversion of a vector  $\mathbf{v}$  in the local coordinates to its counterpart  $\mathbf{V}$  in the global coordinates can be easily done via the rotation matrix  $\mathbf{R}$  as  $\mathbf{V} = \mathbf{R}\mathbf{v}$ . The tangential vector  $\mathbf{e}_\theta$  indicates the direction along which parameter  $\theta$  increases with angle  $\phi$  unchanged; and similarly  $\mathbf{e}_\phi$  points to the direction of increasing  $\phi$  at constant  $\theta$ .

### 2.2. Update process of surface point locations

Now we consider the minimum distance problem for the two ellipsoids  $\mathcal{E}_1$  and  $\mathcal{E}_2$  shown in Fig. 2. The surface-sliding method involves an iterative process, during which the two surface points,  $\mathbf{P}_1$  on  $\mathcal{E}_1$  and  $\mathbf{P}_2$  on  $\mathcal{E}_2$ , update their positions simultaneously to reduce the segment distance between them. For convenience, hereafter, we use the subscript to indicate the ellipsoid and the superscript for the iteration step. For example,  $\mathbf{P}_i^k$  at  $(\theta_i^k, \phi_i^k)$  refers the surface point location on ellipsoid  $\mathcal{E}_i$  ( $i = 1$  or  $2$ ) at iteration  $k$ , and  $k = 0$  denotes the initial parameter values before the iteration starts. Joining  $\mathbf{P}_1^k$  and  $\mathbf{P}_2^k$  by a line segment, we have the direction vector along the connecting line as

$$\mathbf{d}_{12} = -\mathbf{d}_{21} = \mathbf{P}_2^k - \mathbf{P}_1^k. \quad (7)$$

The connecting line can be visualized as an elastic string, and the tension force tends to pull the two end points together till the minimum-distance configuration is reached. To mimic this effect, we project the direction vector  $\mathbf{d}_{ij}$  on to the tangential directions

$$\delta_{\theta,i} = \mathbf{d}_{ij} \cdot \mathbf{e}_{\theta,i}, \quad \delta_{\phi,i} = \mathbf{d}_{ij} \cdot \mathbf{e}_{\phi,i}. \quad (8)$$

Their relative magnitudes indicate in which direction we should move the surface point to reduce the distance. By introducing an angle increment step  $\lambda_i$ , we calculate the respective increment amounts  $d\theta_i$  and  $d\phi_i$  in the  $\mathbf{e}_{\theta,i}$  and  $\mathbf{e}_{\phi,i}$  directions as

$$d\theta_i = \frac{\delta_{\theta,i}}{(\delta_{\theta,i}^2 + \delta_{\phi,i}^2)^{1/2}} \lambda_i, \quad d\phi_i = \frac{\delta_{\phi,i}}{(\delta_{\theta,i}^2 + \delta_{\phi,i}^2)^{1/2}} \lambda_i. \quad (9)$$

The new surface point location  $\mathbf{P}_i^{k+1}$  is then given by

$$\theta_i^{k+1} = \theta_i^k + d\theta_i, \quad \phi_i^{k+1} = \phi_i^k + d\phi_i. \quad (10)$$

The above described calculations should be performed twice, one to update  $\mathbf{P}_1^k$  to  $\mathbf{P}_1^{k+1}$  on ellipsoid  $\mathcal{E}_1$  ( $i = 1$  and  $\mathbf{d}_{ij} = \mathbf{d}_{12}$ ), and another one to update  $\mathbf{P}_2^k$  to  $\mathbf{P}_2^{k+1}$  on ellipsoid  $\mathcal{E}_2$  ( $i = 2$  and  $\mathbf{d}_{ij} = \mathbf{d}_{21}$ ). We thus finish the iteration round, and this process can be repeated till the stopping criteria are satisfied, which will be discussed next.

### 2.3. Implementation details

Other than the iteration calculations outlined above, there are a few implementation issues to be considered for adopting the surface-sliding method in practical computations:

**Initial Locations of  $\mathbf{P}_1^0$  and  $\mathbf{P}_2^0$ .** The initial locations of the two surface points are required so we can start the first iteration. One option could be using the intersection points of the center-to-center connecting line to the two ellipsoidal surfaces. In particulate flow simulations and other similar applications, the relative ellipsoid position and orientation are dynamically updated. In such situations, the search can start with the minimum-distance surface points at the previous simulation time step. Since the change in particle configuration cannot be large over one time step, the search for the new minimum-distance locations should be fast. In our numerical tests below, however, the initial positions of the surface points are assigned individually to challenge the robustness of the algorithm.

**Angle Increment Step  $\lambda_i$ :** The angle increment step  $\lambda_i$  in Eq. (9) plays a similar role as the search step in optimization [20]. In general, a large  $\lambda_i$  can speed up the search process; however, the calculation may become unstable and fail to find the minimum-distance points. On the other hand, a small  $\lambda_i$  is helpful for numerical stability and robustness; but more iterations will be needed to reach the accuracy requirement. In addition, strategies for adaptive step adjustment like in other numerical methods [20] should be implemented to refine the step size in later stages of the search and thus a good accuracy can be achieved. In our following demonstration examples, we set  $\lambda_1^0 = \lambda_2^0$  at the beginning and different values are tested. As for the step refinement, we detect the overshoot situation if  $|\mathbf{d}_{12}^k| \geq |\mathbf{d}_{12}^{k-1}|$ , i.e., the distance does not decrease in this iteration round  $k$ . When overshoot occurs, we halve  $\lambda_1$  and  $\lambda_2$  alternately. This simple strategy works well in our tests, although more delicate methods can be considered [21].

**Stopping Criteria:** Several quantities can be considered as candidates for the stopping criteria to terminate the iteration process. The first one can be the relative error of the  $\mathbf{P}_1 - \mathbf{P}_2$  distance:

$$\epsilon_d = \frac{|\mathbf{d}_{12}^k| - |\mathbf{d}_{12}^{k-1}|}{|\mathbf{d}_{12}^k|}. \quad (11)$$

For a converging search process,  $\epsilon_d$  decreases and gradually approaches 0 at the minimum-distance locations. A threshold value can be set and the results at the end of the last iteration will be accepted as the solution of the problem.

Recall that the  $\mathbf{P}_1 - \mathbf{P}_2$  connecting line should be aligned to the ellipsoid normal directions  $\mathbf{n}_1$  and  $\mathbf{n}_2$  when the minimum distance is found. The angles between them can thus serve as an indicator for the closeness to the minimum-distance solution as well:

$$\epsilon_n = \max \left( 1 - \hat{\mathbf{d}}_{12} \cdot \mathbf{n}_1, \quad 1 - \hat{\mathbf{d}}_{12} \cdot \mathbf{n}_2 \right), \quad \hat{\mathbf{d}}_{12} = \frac{\mathbf{d}_{12}}{|\mathbf{d}_{12}|}. \quad (12)$$

This method has been used in the moving-ball method [12].

Finally, the angle increment step  $\lambda_i$  can also be used to evaluate the converging process. As the search process approaches the minimum-distance solution,  $\lambda_i$  is reduced gradually. A small  $\lambda_i$  value suggests that the current iteration has reached the close vicinity of the solution, and thus we can have

$$\epsilon_\lambda = \max(\lambda_1, \lambda_2) \quad (13)$$

as a criterion for terminating the calculation as well. No specific stopping criterion is applied in our example calculations below (we simply run the simulation for  $K = 500$  iterations and found all cases have converged well at the end.); however, the evolution of these converging indicators will be examined.

### 2.4. Algorithm summary and pseudo-code

For ease of implementation, the complete surface-sliding procedure is summarized in Algorithm 1. Each iteration consists of three stages: (i) the connecting vector between the current surface points is computed and projected onto the local tangential directions (Eqs. (7) and (8)); (ii) the surface parameters and the corresponding surface point locations are updated (Eqs. (9) and (10)); and (iii) the overshoot detection and the adaptive step refinement are applied. When an overshoot is detected, i.e., the separation distance is not decreasing during the present iteration, the update is rejected by restoring the surface points to their previous locations, and the angle increment steps  $\lambda_1$  and  $\lambda_2$  are halved alternately, as tracked by the indicator  $m$  in Algorithm 1.

The iteration terminates when the convergence indicator, selected from Eqs. (11)–(13) or defined similarly as that in Section 3.5, falls below a prescribed tolerance  $\epsilon_{tol}$ , or when a maximum iteration number  $K$  ( $K = 500$  in the following demonstration examples) is reached. At exit, the minimum distance, the closest surface points, and their parametric locations are all directly available, with no additional reconstruction step required. For clarity, Algorithm 1 presents the regular minimum-distance search only; the treatment for the contact and overlap situations discussed in Section 4 can be inserted after Line 14 when such configurations are expected in applications. A GitHub project has also been created and the MATLAB program used for this paper can be downloaded there (see link at the end of this article).

### 2.5. Numerical complexity

The computational cost of the surface-sliding method can be quantified precisely, since every iteration of Algorithm 1 consists of a fixed sequence of closed-form elementary operations. For each of the two ellipsoids, one iteration requires four trigonometric evaluations (sin and cos of  $\theta_i$  and  $\phi_i$ , shared by Eqs. (3)–(6)), the assembly of the surface point, normal, and the two tangent vectors, four  $3 \times 3$  matrix-vector products for the conversion to the global frame, and a small number of dot products and normalizations for the projections in Eq. (8) and the update in Eqs. (9) and (10). In total, one complete iteration involves approximately 300 multiply-add operations (dominated by the eight matrix-vector products), eight trigonometric evaluations, and about ten square roots. The two inverse-cosine evaluations associated with the stopping test can be avoided altogether by comparing  $|\hat{\mathbf{d}}_{ij} \cdot \mathbf{n}_i|$  directly against the precomputed value  $\cos \gamma_e$ , in the same spirit as the reformulation suggested in Ref. [1]. Notably, no linear system needs to be solved, no polynomial root extraction is involved, and the control flow contains no data-dependent sub-algorithm. Overall, the cost per iteration is a small, machine-independent constant, identical for every iteration and – in contrast with curvature- or construction-based methods – entirely insensitive to the ellipsoid sizes, shapes, and orientations. The rotation matrices  $\mathbf{R}_1$  and  $\mathbf{R}_2$  are assembled once per distance query; in dynamic simulations they are, moreover, shared by all pair queries involving the same particle at a given time step.

**Table 1**  
System parameters used for the demonstration examples in this research.

| Particle Parameters | System I<br>(Fig. 3a)                                                      | System II - aligned<br>(Fig. 5a)                     | System II - rotated<br>(Fig. 5b)                         | System III<br>(Fig. 6)                                        |
|---------------------|----------------------------------------------------------------------------|------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------|
| $\mathcal{E}_1$     | $(a_1, b_1, c_1)$<br>$\mathbf{X}_{0,1}$<br>$(\alpha_1, \beta_1, \gamma_1)$ | (1, 0.6, 0.4)<br>(-1.5, 0, 0)<br>(0, $\pi/6$ , 0)    | (1, 0.6, 0.4)<br>(-1.0607, 0, -1.0607)<br>(0, 0, 0)      | (0.2, 0.4, 0.6)<br>(-1.0607, 0, -1.0607)<br>(0, $-\pi/4$ , 0) |
| $\mathcal{E}_2$     | $(a_2, b_2, c_2)$<br>$\mathbf{X}_{0,2}$<br>$(\alpha_2, \beta_2, \gamma_2)$ | (0.6, 0.7, 0.5)<br>(1, 0.5, 0.5)<br>(0, 0, $\pi/4$ ) | (1, 0.6, 0.4)<br>(1.0607, 0, 1.0607)<br>(0, $\pi/4$ , 0) | see text<br>(1.0607, 0, 1.0607)<br>(0, $\pi/4$ , 0)           |

**Algorithm 1** The surface-sliding method for finding the minimum distance between two ellipsoids  $\mathcal{E}_1$  and  $\mathcal{E}_2$ .

**Require:** For each ellipsoid  $i = 1, 2$ : semi-axes  $(a_i, b_i, c_i)$ , center  $\mathbf{X}_{0,i}$ , Euler angles  $(\alpha_i, \beta_i, \gamma_i)$ , and initial surface parameters  $(\theta_i^0, \phi_i^0)$ ; initial angle increment step  $\lambda^0$ ; selected convergence indicator  $\epsilon_d, \epsilon_n$  or  $\epsilon_\lambda$ ; tolerance  $\epsilon_{tol}$ ; maximum iteration number  $K$

**Ensure:** Minimum distance  $d_{min}$ , closest points  $\mathbf{P}_1$  and  $\mathbf{P}_2$ , and their parameters  $(\theta_1, \phi_1)$  and  $(\theta_2, \phi_2)$

```

1:  $k \leftarrow 0$ ;  $\lambda_1 \leftarrow \lambda^0$ ;  $\lambda_2 \leftarrow \lambda^0$ ;  $m \leftarrow 1$   $\triangleright m$  indicates which  $\lambda$  to
   halve next
2: for  $i = 1, 2$ : compute  $\mathbf{P}_i^0$  from  $(\theta_i^0, \phi_i^0)$  via Eqs. (1) and (3)
3:  $d^0 \leftarrow |\mathbf{P}_2^0 - \mathbf{P}_1^0|$ 
4: while  $k < K$  and the stopping criterion is not met [Eq. (11), (12),
   (13), or (17)] do
5:    $\mathbf{d}_{12} \leftarrow \mathbf{P}_2^k - \mathbf{P}_1^k$ ;  $\mathbf{d}_{21} \leftarrow -\mathbf{d}_{12}$   $\triangleright$  Eq. (7)
6:   for  $i = 1, 2$  do
7:     compute  $\mathbf{e}_{\theta,i}$  and  $\mathbf{e}_{\phi,i}$  at  $(\theta_i^k, \phi_i^k)$  via Eqs. (5) and (6), and
     convert them to the global frame using  $\mathbf{R}_i$ 
8:      $\delta_{\theta,i} \leftarrow \mathbf{d}_{ij} \cdot \mathbf{e}_{\theta,i}$ ;  $\delta_{\phi,i} \leftarrow \mathbf{d}_{ij} \cdot \mathbf{e}_{\phi,i}$   $\triangleright$  Eq. (8)
9:     compute  $d\theta_i$  and  $d\phi_i$  from  $\delta_{\theta,i}$ ,  $\delta_{\phi,i}$ , and  $\lambda_i$   $\triangleright$  Eq. (9)
10:     $\theta_i^{k+1} \leftarrow \theta_i^k + d\theta_i$ ;  $\phi_i^{k+1} \leftarrow \phi_i^k + d\phi_i$   $\triangleright$  Eq. (10)
11:    wrap  $\theta_i^{k+1}$  into  $[0, 2\pi)$  if necessary
12:    compute  $\mathbf{P}_i^{k+1}$  from  $(\theta_i^{k+1}, \phi_i^{k+1})$  via Eqs. (3) and (1)
13:  end for
14:   $d^{k+1} \leftarrow |\mathbf{P}_2^{k+1} - \mathbf{P}_1^{k+1}|$ 
15:  if  $d^{k+1} \geq d^k$  then  $\triangleright$  overshoot detected
16:     $(\theta_i^{k+1}, \phi_i^{k+1}) \leftarrow (\theta_i^k, \phi_i^k)$  and  $\mathbf{P}_i^{k+1} \leftarrow \mathbf{P}_i^k$  for  $i = 1, 2$ ;  $d^{k+1} \leftarrow d^k$ 
     $\triangleright$  reject the update (rollback)
17:     $\lambda_m \leftarrow \lambda_m/2$ ;  $m \leftarrow 3 - m$   $\triangleright$  halve  $\lambda_1$  and  $\lambda_2$  alternately
18:  end if
19:   $k \leftarrow k + 1$ 
20: end while
21: return  $d_{min} = d^k$ ,  $\mathbf{P}_1^k$ ,  $\mathbf{P}_2^k$ ,  $(\theta_1^k, \phi_1^k)$ , and  $(\theta_2^k, \phi_2^k)$ 

```

The total time complexity is therefore  $O(N_{it})$  with the above constant, where  $N_{it}$  is the number of iterations to convergence. The iteration count consists of an approach stage, proportional to the parametric path length between the initial and final surface points divided by the initial step  $\lambda^0$ , followed by a refinement stage governed by the alternate step halving. Since each rejected update halves one of the angle increments, the refinement stage grows only logarithmically with the accuracy requirement. This scaling is confirmed by the benchmark study of Section 3.5: tightening the error bound  $\epsilon_{d^*}/D_{eq}$  from  $10^{-4}$  to  $10^{-6}$  and  $10^{-8}$  increases the iteration count from 25~36 to 48~61 and 65~74, respectively, i.e., by roughly twenty iterations per two orders of magnitude in accuracy, uniformly over all tested aspect ratios and scales. In particle-resolved simulations, where the search can be warm-started from the minimum-distance points of the previous time step (Section 2), the approach stage is only necessary at the beginning of the simulation and the logarithmic refinement dominates.

The memory footprint is  $O(1)$  and very small: a distance query requires only the current and previous parameter pairs  $(\theta_i, \phi_i)$ , the two step sizes with the alternation indicator, the two rotation matrices and

center positions, and a fixed set of intermediate three-element vectors — fewer than one hundred scalar values in total, with no dynamically allocated storage and no history. For comparison, the GJK algorithm maintains and updates a simplex vertex set together with cached dot products and determinant coefficients for its distance sub-algorithm [1, 14], the moving-ball method solves a quadratic segment-surface intersection per ellipsoid per iteration [1, 12], and algebraic formulations operate on high-degree polynomial coefficients whose root extraction can be numerically delicate in floating-point arithmetic [17, 18]. The fixed, branch-light arithmetic and constant memory of the surface-sliding iteration make it straightforward to vectorize and well suited to the large numbers of simultaneous pair queries encountered in particle-resolved simulations of ellipsoid suspensions [1].

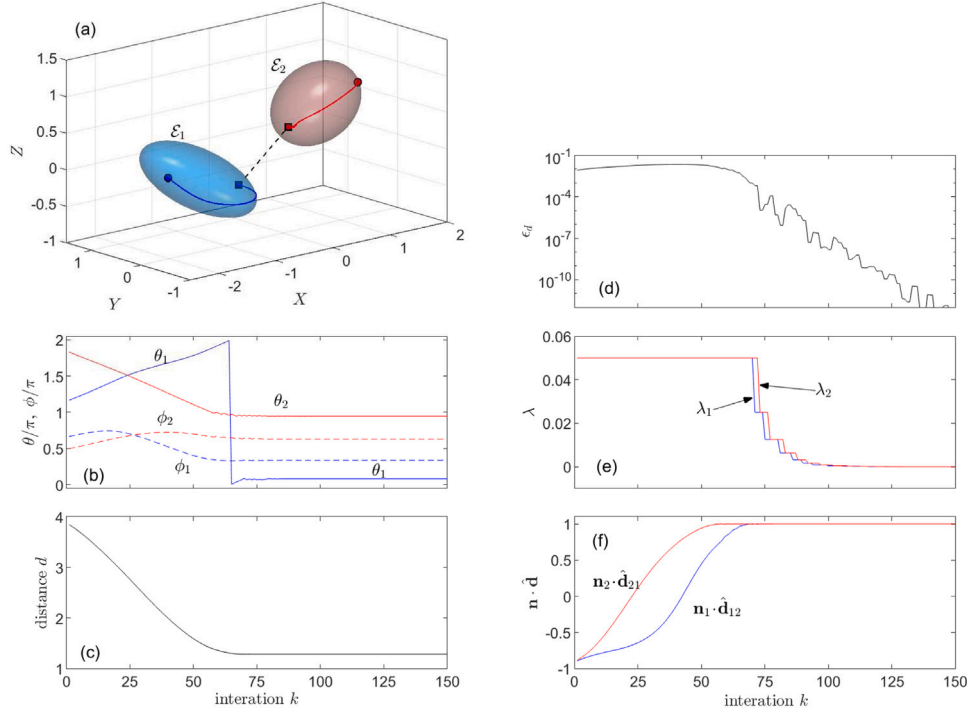
### 3. Demonstration examples and benchmark validation

In this section, we first present several demonstration examples with ellipsoids of various locations, orientations, sizes and aspect contrasts. Different initial positions and search steps are also tested to investigate the algorithm robustness. In Section 3.5, our surface-sliding method is tested using the benchmark system from Ref. [1] and its performance is compared to other methods.

#### 3.1. Converging process

The first case we test is the configuration shown in Fig. 3a. The dimensions, positions and orientations of the two ellipsoids are listed in Table 1 (System I). The search starts with surface points at  $(\theta_1^0, \phi_1^0) = (7\pi/6, 2\pi/3)$  on  $\mathcal{E}_1$  and  $(\theta_2^0, \phi_2^0) = (11\pi/6, \pi/2)$  on  $\mathcal{E}_2$ . These initial locations actually are on the opposite sides of those of the minimum-distance solution, and this poses more challenge to the algorithm than using the center-to-center intersection points. The angle increment steps are  $\lambda_1^0 = \lambda_2^0 = 0.05$ . The trajectories of the surface points during the search process can be visualized in Fig. 3a, where the circles represent the starting points and the squares are the minimum-distance pair. More quantitatively, Fig. 3b shows the evolution of surface point location parameters. The sudden jump in  $\theta_1$  occurs since, at iteration  $k = 65$ ,  $\theta_1$  increases over  $2\pi$  and it is set back to the  $0 \sim 2\pi$  definition range of  $\theta$ . It can be seen there that, once the search commences, all the position parameters start to change under the *tension* of the connecting line, and they reach their individual steady values quickly after ~75 iterations. Meanwhile, the distance between the two surface points continuously decreases and a steady value is established at  $d_{min}^* = 1.2856$ . This value is considered as the minimum distance between the ellipsoids in Fig. 3a in our next analysis.

The converging process is further illustrated in Figs. 3d-f: The relative distance error  $\epsilon_d$  stays relatively constant in  $0.01 \sim 0.02$  in the early stage  $k = 1 \sim 60$ ; however, it quickly drops later, to  $< 10^{-10}$  for  $k > 120$ . Simultaneously, the refinement process for the angle increment steps  $\lambda_1$  and  $\lambda_2$  kicks off at  $k \approx 70$  as the surface points have moved to the vicinity of the minimum-distance solution. The search steps reduce quickly to  $\sim 10^{-6}$  at  $k > 120$ , suggesting that our algorithm has successfully found the minimum-distance spots and it is just refining for a more accurate final result. Moreover, the dot products  $\mathbf{n}_i \cdot \hat{\mathbf{d}}_{ij}$  in Fig. 3f quickly increase to 1, indicating that the connecting



**Fig. 3.** The search process for System I in Table 1: (a) a 3D view of the relative positions and orientations of the two ellipsoids, (b) the evolution of the surface point locations, (c) the converging process in distance between the surface points, (d) the relative error in separation distance, (e) the refining process of the angle increment steps, and (f) the improving alignment between the surface normal directions and the connecting segment. In (a), the circles represent the initial surface point positions and the squares for their final locations when the minimum distance is found, and the curves over the ellipsoid surfaces are the surface point trajectories.

segment aligns nearly perfectly to the local normal directions and the correct minimum-distance situation is found. Please note that this requirement is not directly involved in our search calculation, but automatically satisfied as the iteration converges. Also their initial values are approximately  $-1$ , meaning the initial condition for our search algorithm is relatively harsh with the outward normals pointing in the opposite directions to the *tension* from the connecting segment. This demonstrates the robustness of our algorithm in dealing with unfavorable search conditions.

### 3.2. Initialization insensitivity

The same configuration in Fig. 3a is re-tested in this subsection with different initial conditions. Starting with the same initial positions  $(\theta_1^0, \phi_1^0) = (7\pi/6, 2\pi/3)$  and  $(\theta_2^0, \phi_2^0) = (11\pi/6, \pi/2)$ , different values for the initial angle increment step  $\lambda^0$  are utilized in Fig. 4a. The observation is consistent with the general behaviors of search step influence in other numerical methods. The large step value  $\lambda^0 = 0.5$  causes oscillations at the beginning; however, the search quickly converges. All other  $\lambda^0$  values yield smooth, monotonically decreasing curves for distance  $d$ . It takes more iterations for small  $\lambda^0$  values, which is reasonable since a smaller search step requires more iterations to approach the correct solution. The minimum distance  $d_{min}^* = 1.2856$  is considered as the correct solution based on our analysis in the previous section, and it is plotted in Fig. 4a for comparison. Clearly, all calculations converge to this value accurately, and this further confirms that  $d_{min}^* = 1.2856$  is indeed the solution for the system considered here.  $\lambda^0 = 0.05$  will be used in all next calculations.

In addition, we test other three different sets of initial surface point locations, denoted as Cases B–D in Fig. 4b with Case A for that used Figs. 3 and 4a above. The converging processes of separation distance  $d$  is shown in Fig. 4b, while their search paths are displayed in the  $\theta - \phi$  plot in Fig. 4c. The same line styles are adopted in Figs. 4b and

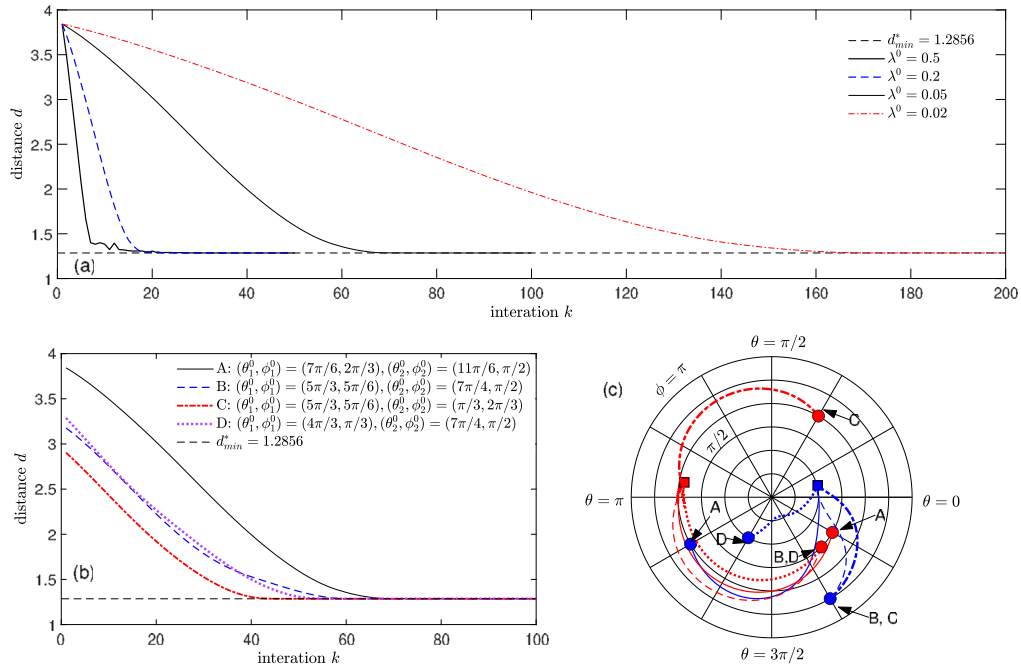
c for clarity, and the individual starting locations of these cases are labeled in Fig. 4c. Although they start from different initial positions and follow different search routes, the same minimum-distance solution is found in  $\sim 70$  iterations, as indicated by the same converged distance  $d_{min}^* = 1.2856$  in Fig. 4b and the same end points (squares) for all search paths in Fig. 4c. It takes a little longer for Case A and a little shorter for Case C for converging to  $d_{min}^*$ , since the separation distance  $d$  is the largest at the beginning in Case A (3.8427) and the least in Case C (2.9038). These results indicate that the surface-sliding algorithm can handle different initial search conditions and find correct solutions.

### 3.3. Spatial isotropy

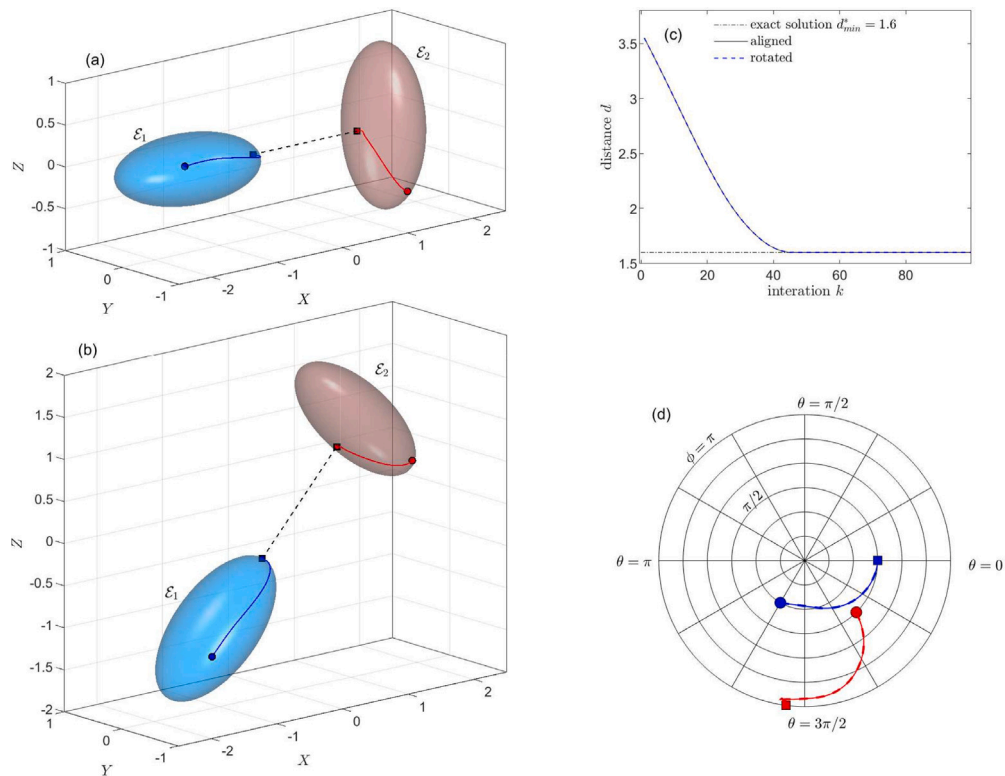
The next comparison is carried out to test if the surface-sliding method satisfies the spatial isotropy requirement, meaning that the minimum-distance solution should be determined by the relative positions and orientations of the two ellipsoids, and independent of the artificial selection of the global coordinate system. The two configurations considered are displayed in Figs. 5a and b, and their respective parameters are given in Table 1 (Systems II — aligned and II — rotated). It can be seen that the system in Fig. 5b is just a rotation of that in Fig. 5a about the coordinate axis  $Y$ , and thus the same search process, including the converging path in the  $\theta - \phi$  plot and the final minimum-distance solution, should be identical. Also, for the particular setup in Fig. 5a, the minimum distance occurs between the rightmost point on  $\mathcal{E}_1$  at  $(\theta_1^* = 0, \phi_1^* = \pi/2)$  and the leftmost point on  $\mathcal{E}_2$  at its south pole ( $\phi_2^* = \pi$  and  $\theta_2^*$  is not defined or arbitrary); and the minimum distance value is available analytically as

$$d_{min}^* = |\mathbf{X}_{0,2} - \mathbf{X}_{0,1}| - a_1 - c_2 \quad (14)$$

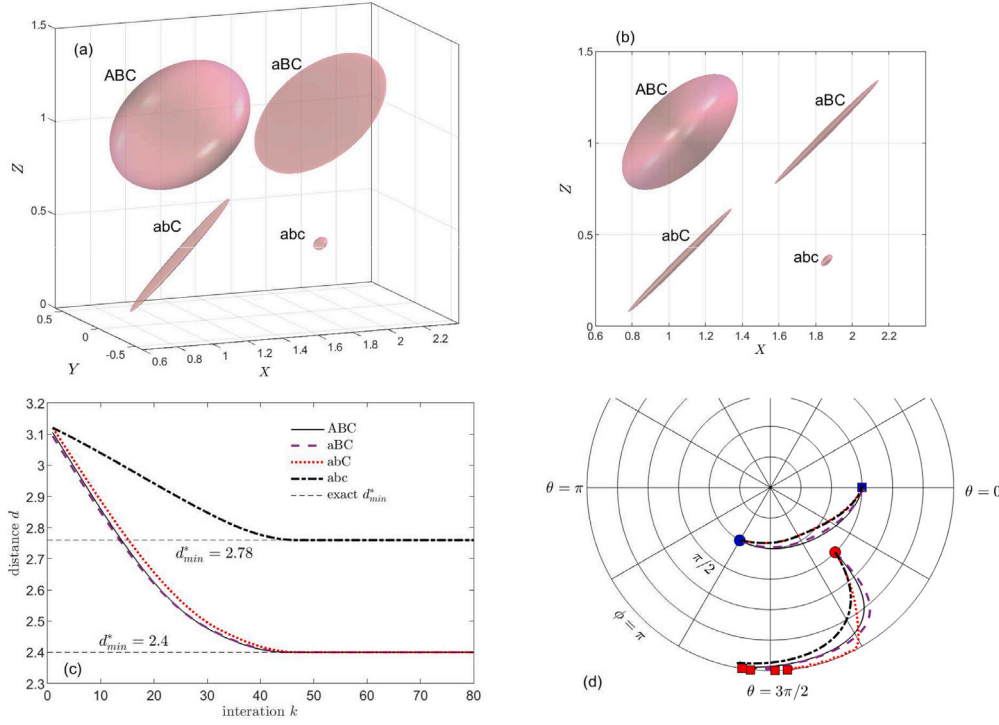
With  $|\mathbf{X}_{0,2} - \mathbf{X}_{0,1}| = 3$ ,  $a_1 = 1$  and  $c_2 = 0.4$  for both systems, we thus have  $d_{min}^* = 1.4$ . The final search result, both the separation distance in Fig. 5c and the surface point locations in Fig. 5d, matches the above



**Fig. 4.** Influences of (a) different initial angle increment step  $\lambda^0$  and (b and c) different initial positions of the surface points. In (c), the circles represent the initial surface point positions at the beginning of the search, the squares represent their final positions when the solution is found, and the curves between them indicate the search paths in the  $\theta - \phi$  plane. Different colors are used: blue for ellipsoid  $\mathcal{E}_1$  and red for ellipsoid  $\mathcal{E}_2$ . The same line styles are adopted in (b) and (c) for each case for clarity.



**Fig. 5.** 3D views for System II in Table 1 (a for the aligned configuration and b for the rotated configuration). The circles represent the initial surface point positions and the squares for their final locations when the minimum distance is found, and the curves over the ellipsoid surfaces are the surface point trajectories. The search processes with these two configurations are compared in (c) for the separation distance  $d$  and (d) for the search paths in the  $\theta - \phi$  plane.



**Fig. 6.** Effects of ellipsoid size and geometry on the search process. The (a) 3D and (b)  $X-Z$  views for the four different shapes of  $\mathcal{E}_2$  are displayed together for comparison. Detailed parameters are provided in Table 1 (System III) and in the text. The converging processes in separation distance  $d$  are given in (c) with the exact solutions  $d_{min}^*$  shown as dashed lines for comparison. The search paths in the  $\theta-\phi$  plane are also presented in (d), using the same line styles as in (c) for clarity.

predictions perfectly. Also, excellent agreement is noted in both the distance converging processes and the search paths in the  $\theta-\phi$  plot for these two aligned and rotated setups, with no visual difference between the curves. This comparison convincingly illustrates that our method can yield reliable results independent of the global coordinate frame selection.

### 3.4. Robustness with particle size and shape

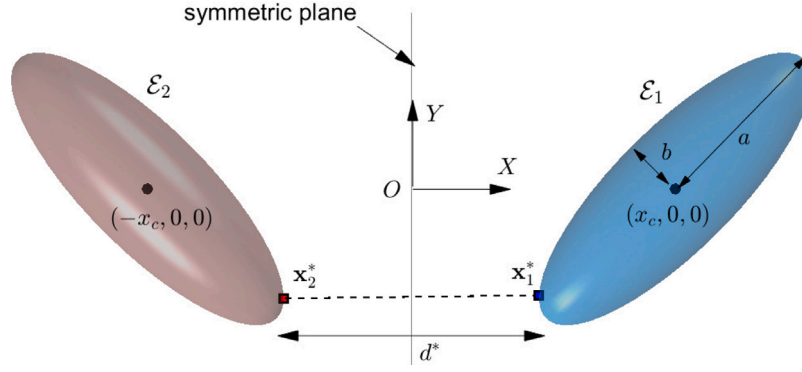
It has been reported that, for the moving-ball method, the convergence performance is less satisfactory for ellipsoids with large contrast in size and dimensions [1,12]. To evaluate our algorithm on this aspect, we use the rotated system in Fig. 5b, however, with modified ellipsoid dimensions: The  $\mathcal{E}_1$  semi-axes  $(a_1, b_1, c_1) = (0.2, 0.4, 0.6)$ , while four different  $\mathcal{E}_2$  dimensions are considered. We denote the  $\mathcal{E}_2$  shape identical to  $\mathcal{E}_1$  with  $(a_2, b_2, c_2) = (0.2, 0.4, 0.6)$  as Shape ABC, and label the other three shapes as: Shape aBC with  $(a_2, b_2, c_2) = (0.02, 0.4, 0.6)$ , Shape abC with  $(a_2, b_2, c_2) = (0.02, 0.04, 0.6)$ , and Shape abc with  $(a_2, b_2, c_2) = (0.02, 0.04, 0.06)$ . From Shapes ABC to abc, we reduce one additional semi-axis length by a factor of ten at every step. The reduced dimensions are highlighted in bold, and we use uppercase letters from the original length and the lowercase letters for the reduced dimension in the shape labels. These shapes are compared in Figs. 6a and b, and the geometric contrast among them is distinctive. We can see there that Shape aBC represents a thin elliptical disk, Shape abC resembles a thin oblong blade, and obviously Shape abc is just a much smaller version of Shape ABC. The expression in Eq. (14) is still valid, which yields  $d_{min}^* = 2.4$  for Shapes ABC, aBC and abC and  $d_{min}^* = 2.78$  for Shape abc. The minimum-distance points are still at  $(\theta_1^* = 0, \phi_1^* = \pi/2)$  on  $\mathcal{E}_1$  and the south pole of  $\mathcal{E}_2$  at  $\phi_2^* = \pi$ .

We start the search with the same initial positions  $(\theta_1^0 = 4\pi/3, \phi_1^0 = \pi/3)$  and  $(\theta_2^0 = 7\pi/4, \phi_2^0 = \pi/2)$ , and use the same initial angle increment  $\lambda^0 = 0.05$  for all the four  $\mathcal{E}_2$  shapes. The distance evolution

and surface point trajectories during the search processes are displayed in Figs. 6c and d, respectively. For all cases, the correct minimum distance is quickly found in  $\sim 50$  iterations, and the final surface point locations (squares in Fig. 6d) agree with our expectation excellently. Please note that the four red squares in Fig. 6d actually represent the same search destination on  $\mathcal{E}_2$  - the south pole with  $\phi_2^* = \pi$  and any arbitrary  $\theta_2$  values. Unlike in the moving-ball method [1,12], the geometric aspect contrast has no apparent influence on the stability, accuracy and converging speed in our method, and there is no need to adjust the initial angle increment  $\lambda^0$  for different ellipsoid dimensions. This favorable feature is due to the fact that our search process actually proceeds in the  $\theta-\phi$  space, which is not directly related to the ellipsoid size or shape. On the contrary, the moving-ball method relies on the inner sphere radius to find the next surface location, and hence the ellipsoid size and local curvature affect the search performance.

### 3.5. Benchmark validation and comparison with existing methods

To enable a direct and quantitative comparison with existing techniques, we evaluate the surface-sliding method on the analytical benchmark introduced by Girault et al. [1], where the moving-ball (MB) [12], Gilbert-Johnson-Keerthi (GJK) [13,14], Newton-Coulomb (NC) [15], and exterior penalty function (EPF) [16] methods were assessed under identical conditions. The configuration consists of a spheroid  $\mathcal{E}_1$  with semi-axes  $(a, b, b)$  and the equivalent diameter is defined as  $D_{eq} = (ab^2)^{1/3}$ , such that  $a = A_r^{2/3} D_{eq}/2$  and  $b = A_r^{-1/3} D_{eq}/2$  for an aspect ratio  $A_r = a/b$ .  $\mathcal{E}_1$  is centered on the positive  $X$  axis and rotated by  $\pi/4$  about the  $Z$  axis, while  $\mathcal{E}_2$  is its mirror image with respect to the  $YZ$  plane, as shown in Fig. 7. The exact minimum distance  $d^*$  and the exact closest points  $\mathbf{x}_m^*$  are available in closed form from the support mapping of the ellipsoid [1]. They can also be found by simply setting the ellipsoid surface normal to be  $(\mp 1, 0, 0)$  on  $\mathcal{E}_1$  and  $\mathcal{E}_2$ , respectively.



**Fig. 7.** The  $XY$  view of the benchmark validation system in Section 3.5. The two ellipsoids  $\mathcal{E}_1$  and  $\mathcal{E}_2$  are identical with semi-axes  $(a, b, b)$ , locate at  $(\pm x_c, 0, 0)$ , and have rotated about their individual  $z$  axis by  $\pm\pi/4$ , respectively. They are symmetric about the  $YZ$  plane and the minimum distance  $d^*$  occurs between  $\mathbf{x}_1^*$  on  $\mathcal{E}_1$  and  $\mathbf{x}_2^*$  on  $\mathcal{E}_2$  as given in Eqs. (15) and (16).

The analytical expressions for the closest point locations are given as:

$$\mathbf{x}_1^* = \begin{bmatrix} x_c \\ 0 \\ 0 \end{bmatrix} - \frac{1}{\sqrt{2(a^2 + b^2)}} \begin{bmatrix} a^2 + b^2 \\ a^2 - b^2 \\ 0 \end{bmatrix}; \quad \mathbf{x}_2^* = \begin{bmatrix} -x_c \\ 0 \\ 0 \end{bmatrix} - \frac{1}{\sqrt{2(a^2 + b^2)}} \begin{bmatrix} -a^2 - b^2 \\ a^2 - b^2 \\ 0 \end{bmatrix}. \quad (15)$$

and the minimum distance  $d^*$  thus is

$$d^* = |\mathbf{x}_1^* - \mathbf{x}_2^*| = 2x_c - \sqrt{2(a^2 + b^2)}. \quad (16)$$

This allows the center positions to be set for any prescribed value of  $d^*$ . Following Ref. [1], the exact distance is varied over  $d^*/D_{eq} \in \{10^0, \dots, 10^{-6}\}$ , the aspect ratio over  $A_r \in \{1/6, 1/3, 2/3, 3/2, 3, 6\}$ , and a global dilation parameter  $\Lambda \in \{10^{-6}, 10^{-3}, 1, 10^3, 10^6\}$  rescales every length ( $a, b, D_{eq}$  and  $d^*$ ) of the configuration to test scale independence. The search is initialized, as in Ref. [1], at the intersection points of the center-to-center segment with the two surfaces, and the initial angle increment step is kept at  $\lambda^0 = 0.05$  for all 98 queries reported below, with no case-by-case adjustment.

For consistency with the error-controlled comparisons of Ref. [1], the iteration is terminated by an automated stopping rule derived from the same geometric bound employed there. Denoting  $\delta_i = \arccos[\hat{\mathbf{d}}_{ij} \cdot \mathbf{n}_i]$  for the angle between the connecting segment and the surface normal at  $\mathbf{P}_i$ , the calculation stops when

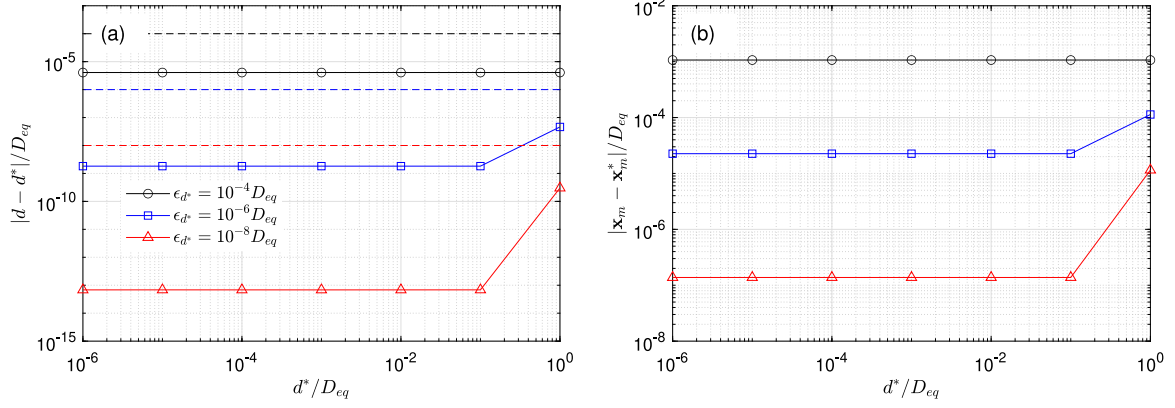
$$\max(\delta_1, \delta_2) \leq \delta_e = \sqrt{\frac{2\epsilon_{d^*}}{R_{max,1} + R_{max,2}}}, \quad R_{max,i} = \frac{[\max(a_i, b_i, c_i)]^2}{\min(a_i, b_i, c_i)}, \quad (17)$$

where  $R_{max,i}$  is the global maximum curvature radius of ellipsoid  $\mathcal{E}_i$  and  $\epsilon_{d^*}$  is the prescribed bound on the distance error. This choice guarantees  $|d - d^*| \leq \epsilon_{d^*}$  through the relation  $|d - d^*| \leq \delta_e^2(R_{max,1} + R_{max,2})/2$  [1]. Please note that the normal angle  $\delta_i$  can be related to the error indicator in Eq. (12) for small  $\delta_i$  values in the refinement stage as  $\epsilon_n \approx \max(\delta_1^2/2, \delta_2^2/2)$ . In addition, the overshoot detection described in Section 2 is taken in its non-strict form: an attempted update is rejected (and the step refinement triggered) whenever  $|\mathbf{d}_{12}^k| \geq |\mathbf{d}_{12}^{k-1}|$ , i.e., whenever the move fails to strictly reduce the separation.

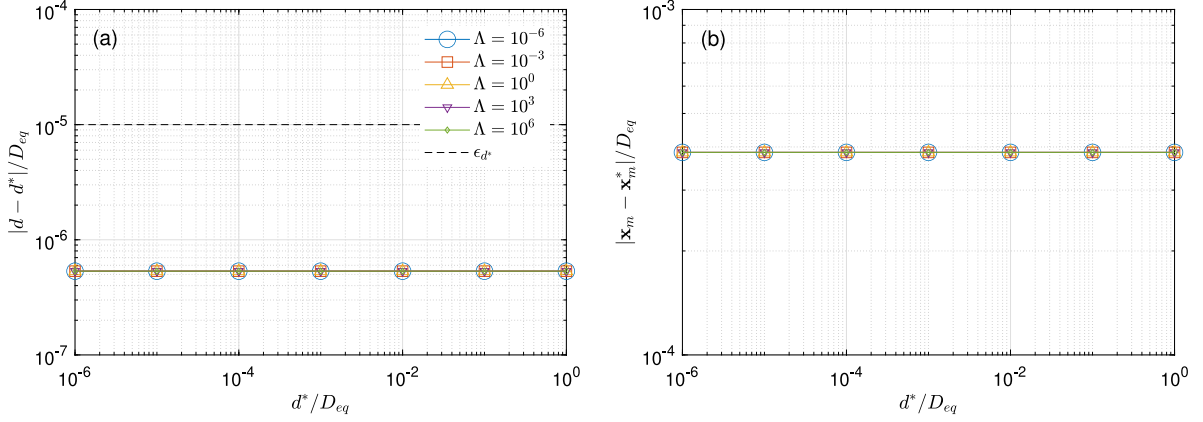
The results of the three parameter sweeps are presented in Figs. 8–11. All 98 distance queries converge through the criterion in Eq. (17) within 25 to 74 iterations (43 on average), and the prescribed error bound is satisfied in every single case, with a maximum recorded error of  $0.15 \epsilon_{d^*}$ . Fig. 8 confirms that the distance error follows the prescribed tolerance over four orders of magnitude of  $\epsilon_{d^*}$  and over the whole range of separation  $d^*$ , including the near-contact regime  $d^*/D_{eq} = 10^{-6}$  that is critical for lubrication corrections in particle-resolved simulations [1]. The number of iterations grows only mildly as the tolerance is tightened (25, 48, and 65 iterations for  $\epsilon_{d^*}/D_{eq} = 10^{-4}$ ,  $10^{-6}$ , and  $10^{-8}$ , respectively, at  $A_r = 3$ ), reflecting the geometric reduction of

the angle increment steps by the alternate-halving refinement. Fig. 9 addresses scale independence: the normalized error curves obtained for dilation spanning twelve orders of magnitude are identical to machine precision. Unlike methods whose step size carries the dimension of a length, this property is inherent to the surface-sliding formulation, since the search proceeds in the dimensionless  $\theta$ - $\phi$  parameter space and  $\lambda_i$  is an angle-like dimensionless parameter. Finally, Figs. 10 and 11 examine the influence of the aspect ratio: both the achieved accuracy and the iteration count remain essentially flat from the thin oblate disk ( $A_r = 1/6$ ) to the elongated spheroid ( $A_r = 6$ ). Please note that the same  $\lambda^0 = 0.05$  has been used throughout all the benchmark calculations, and the iteration count numbers can be significantly reduced if a larger value is adopted to speed up the approach stage (see Fig. 4). While all cases converge smoothly and meet the accuracy requirement, it is noted that, for few large separation cases with  $d^*/D_{eq} = 10^0$  in Figs. 8–10, the final distance error is slightly larger than other smaller separation cases. This observation is due to the fact that all key control parameters in our search process, such as the separation distance  $d$  and the normal-link angles  $\delta_1$  and  $\delta_2$ , are less sensitive to surface position changes at larger  $d^*$  values. This is evident in Fig. 11, where more iteration steps are required for those few cases. A possible remedy for such slightly slower converging processes is to use a larger increment step  $\lambda^0$ . Fortunately, in practical applications, such minimum distance calculations are only necessary for close particle pairs.

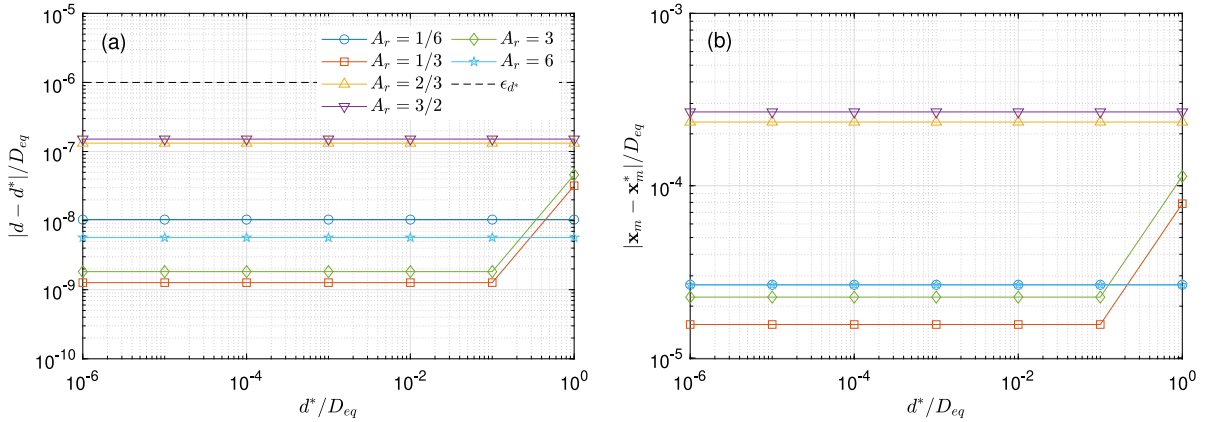
These results can be placed alongside the systematic assessment in Ref. [1], which evaluated the MB, GJK, NC, and EPF methods on the identical benchmark system; and the comparison is summarized in Table 2. Two of the four methods were ruled out there: the NC method required the calibration of several internal parameters, failed to converge for 3 of 606 queries in a subsequent packing test, and consumed over two orders of magnitude more computing time than GJK; the EPF method lost scale independence, could not reach high accuracy for small separations, and was two to four orders of magnitude slower than GJK [1]. The surface-sliding method satisfies all the precision, consistency, and scale-independence criteria as the two retained methods (MB and GJK). However, our method has the closest points and contact normal directly available from the iteration, whereas the GJK algorithm requires a support-mapping formulation, a simplex sub-algorithm, and a dedicated reconstruction step to recover the closest points [1,14]. On the other hand, it was noticed in Ref. [1] that the computing time of the MB method deteriorates by factors above four compared to GJK at  $A_r = 1/6$  in their statistical packing study, while our method exhibits impressive features of aspect-ratio insensitivity and size- and shape-robustness as demonstrated in Figs. 6, 10 and 11. As discussed in Section 3, these favorable behaviors originate from the search being conducted in the parametric space, decoupled from the ellipsoid dimensions, in contrast with the inscribed-sphere construction



**Fig. 8.** Error control on the benchmark system with  $A_r = 3$  and  $\Lambda = 1$ : (a) the normalized distance error and (b) closest-point location error versus the separation distance, for three values of the stopping parameter  $\epsilon_{d^*}$  (dashed lines). The inequality  $|d - d^*| \leq \epsilon_d$  holds for all cases.



**Fig. 9.** Scale independence examination for the benchmark system with  $A_r = 3$  and  $\epsilon_{d^*}/D_{eq} = 10^{-5}$ : (a) the normalized distance error and (b) closest-point location error, for different global dilation factors  $\Lambda = 10^{-6}$  to  $10^6$ . The five curves are identical to machine precision and collapse onto a single line in these plots (nested markers).



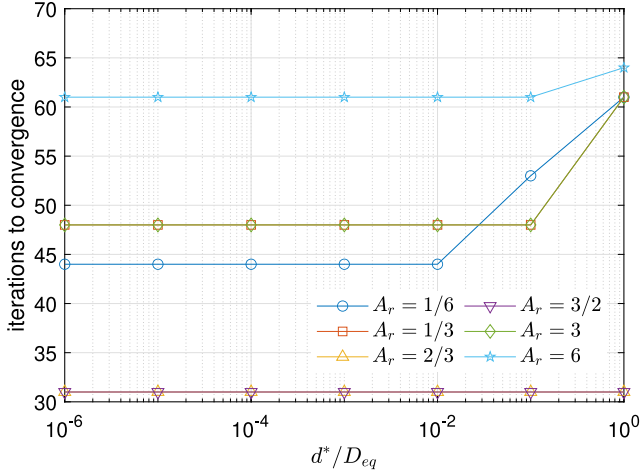
**Fig. 10.** Aspect-ratio robustness examination for the benchmark system with  $\epsilon_{d^*}/D_{eq} = 10^{-6}$  and  $\Lambda = 1$ : (a) the normalized distance error and (b) closest-point location error for aspect ratios from  $A_r = 1/6$  (thin oblate disk) to  $A_r = 6$  (elongated spheroid), with the same initial step  $\lambda^0 = 0.05$  for all cases.

**Table 2**

Qualitative performance comparison for different minimum-distance methods based on the benchmark system. The assessments of the moving-ball (MB), GJK, Newton–Coulomb (NC), and exterior penalty function (EPF) methods are extracted from Ref. [1], and the last column corresponds to the present surface-sliding (SS) method.

| Criterion                                                     | MB             | GJK | NC | EPF | SS (present)      |
|---------------------------------------------------------------|----------------|-----|----|-----|-------------------|
| Controlled distance error $ d - d^*  \leq \epsilon_d$         | ✓              | ✓   | ✓  | ×   | ✓                 |
| Scale independence                                            | ✓              | ✓   | ✓  | ×   | ✓(inherent)       |
| Robustness over $A_r \in [1/6, 6]$                            | ✓ <sup>a</sup> | ✓   | ×  | ×   | ✓                 |
| Accuracy at near contact ( $d^*/D_{eq} \rightarrow 10^{-6}$ ) | ✓              | ✓   | ✓  | ×   | ✓                 |
| Internal parameters to calibrate                              | 0              | 0   | 3  | 2   | 1 ( $\lambda^0$ ) |
| Closest points without reconstruction                         | ✓              | ×   | ✓  | ✓   | ✓                 |

<sup>a</sup> With computing time deteriorating for extreme aspect ratios; see text and Ref. [1].



**Fig. 11.** Iterations to convergence for the aspect-ratio sweep of Fig. 10. The convergence behavior is essentially independent of both the aspect ratio and the separation distance.

of the MB method whose radius collapses for thin shapes. Although the initial angle increment  $\lambda^0$  in our surface-sliding method needs to be calibrated/selected properly and no such calibration work is required in MB and GJK, this is only done once at the beginning of a simulation, and the search process is robust even when a relatively large  $\lambda^0$  is used (see Fig. 4). The associated computation cost is negligible.

#### 4. Summary and concluding remarks

We have proposed a novel surface-sliding method for finding the minimum distance and the corresponding surface point locations between two ellipsoids. The iterative search process takes place in the individual  $\theta - \phi$  space of each ellipsoid, and the search direction is guided by the *tension* along the connecting segment. Several demonstration examples have been designed and benchmark validations against analytical solutions are conducted as well. These calculation results show satisfactory numerical performance of our method, including the stability, accuracy and robustness. Compared to other existing techniques, our method possesses unique features such as the clear geometric concept and simple mathematical formulation. This surface-sliding method is expected to be an attractive choice for computer graphics, engineering design and computational science.

In the method description and numerical examples, we have not considered the situations when the two ellipsoids are in direct contact (Fig. 12a) or they may even overlap each other (Fig. 12b). These situations need to be addressed in some applications such as the discrete element model and material modeling [6,10,22]. This concern can be conveniently incorporated in our method as follows. At the

end of iteration  $k$ , if the distance  $|\mathbf{P}_2^k - \mathbf{P}_1^k|$  is smaller than a threshold value  $\sigma$ , one can consider that the two ellipsoids are either in contact or intersecting each other. By comparing  $\mathbf{n}_1 \cdot \mathbf{n}_2$  versus  $-1$ , we can separate the in-contact (if  $\mathbf{n}_1 \cdot \mathbf{n}_2 \approx -1$ ) condition from the overlap situation. No further treatment is required for the in-contact case. If the overlap (penetration) magnitude (dashed double arrow in Fig. 12b) is needed, we can continue the search from  $\mathbf{P}_1^k$  under the direction of  $-\mathbf{n}_2$  on  $\mathcal{E}_1$ , and similarly from  $\mathbf{P}_2^k$  under the direction of  $-\mathbf{n}_1$  on  $\mathcal{E}_2$ . Once the distance increases back beyond the threshold  $\sigma$ , we can resume the regular update calculation, and gradually approach the maximum overlap locations  $\mathbf{P}_1^*$  and  $\mathbf{P}_2^*$  (Fig. 12b). We would also like to mention that this surface-sliding concept can be readily applied to other scenarios, such as the minimum distance problems between an ellipsoid and a fixed location or a flat wall [7,22], between other parametric surfaces [23], and between a convex particle and the concave wall in a container. These extensions are straightforward, and researchers with such needs can quickly modify the corresponding equations we present above to accomplish their individual tasks.

At last, a few limitations of the present study should be mentioned. First, the performance assessment in Section 3.5 concerns accuracy, consistency, and robustness; a wall-clock computing-time comparison of these different methods requires all methods to be implemented in the same language on the same hardware, and it is thus left outside the scope of this work. The iteration counts in Fig. 11, together with the fixed per-iteration cost quantified in Section 2.5, nevertheless indicate that the computational effort of the surface-sliding method is modest and, importantly, insensitive to the particle geometry. Second, the tangential vector  $\mathbf{e}_\theta$  at the north and south poles (with  $\phi = 0$  or  $\pi$ ) is not well defined in Eq. (5). Although we have not encountered any issues in our calculations, the uncertainty or inaccuracy in  $\mathbf{e}_\theta$  calculation near the poles may cause numerical instability or oscillations in large-scale simulations. To prevent this potential risk, one can *rotate* the local coordinate for the particle, for example,  $y \rightarrow x'$ ,  $z \rightarrow y'$  and  $x \rightarrow z'$  (with semi-axes  $b \rightarrow a'$ ,  $c \rightarrow b'$  and  $a \rightarrow c'$ ) when the  $\phi$  value is too close to 0 or  $\pi$  at the end of one iteration. Here we use the superscript  $'$  to denote the variables associated with the new local coordinate system. After this coordinate rotation, the surface point will be near the equator ( $\phi' = \pi/2$ ) in the new coordinate. In addition, the surface-sliding process relies on a smooth parametric representation of the two surfaces, as required by other methods as well. For discretized surfaces represented by vertices and surface elements, for example, the deformable particles/capsules in fluid flows [24,25], the formulation presented in Section 2 cannot be adopted. Nevertheless, the surface-sliding concept might still be applicable with the surface normal vector obtained from the vertex positions numerically [26].

#### Computer program

The MATLAB program used for the calculations in this article can be downloaded from the following GitHub repository: <https://github.com/DariushAmirkhani/surface-sliding-ellipsoid-distance>.

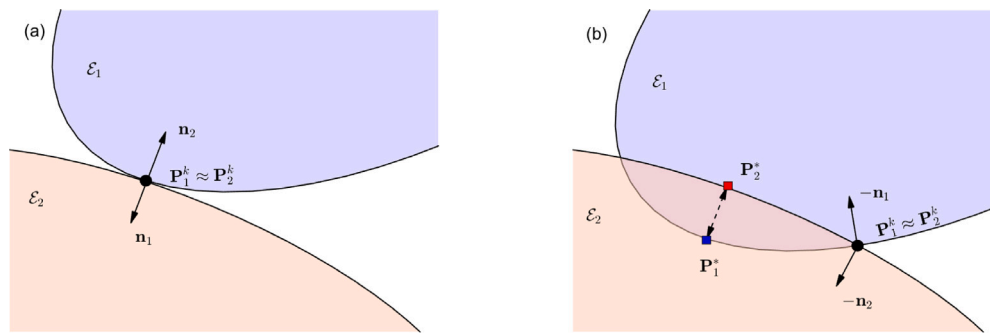


Fig. 12. Two-dimensional illustrations for the (a) in-contact and (b) overlap situations of two ellipsoids.

### CRedit authorship contribution statement

**Dariusz Amirkhani:** Writing – original draft, Visualization, Validation, Software, Investigation, Formal analysis, Data curation, Conceptualization. **Junfeng Zhang:** Writing – review & editing, Visualization, Validation, Supervision, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization.

### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### Acknowledgments

The authors thank the anonymous reviewers for providing thoughtful comments and constructive suggestions. This work was supported by the Natural Science and Engineering Research Council of Canada (NSERC).

### Data availability

No data was used for the research described in the article.

### References

- [1] I. Girault, M.A. Chadil, S. Vincent, Comparison of several numerical algorithms to compute the distance between two ellipsoids, *J. Comput. Phys.* 458 (2022) 111100.
- [2] K. Anoukou, R. Brenner, F. Hong, M. Pellerin, K. Danas, Random distribution of polydisperse ellipsoidal inclusions and homogenization estimates for porous elastic materials, *Comput. Struct.* 210 (2018) 87–101.
- [3] M. Jbeili, J. Zhang, Particle-resolved simulations for nanofluid thermal enhancement in channel flows, *Numer. Heat Transf. - Part A: Appl.* 84 (2023) 1417–1435.
- [4] X. Yan, Y. Hu, Y. Xie, Q.-Z. Fang, Efficient RVE modeling for ellipsoidal particle-and short fiber-hybrid reinforced composites: Novel algorithms for overlap detection and geometric periodicity, *Compos. Struct.* 370 (2025) 119422.
- [5] F. Kamarei, T. Goudarzi, A novel method for investigating the isotropy of response for randomly generated unit cells of particulate composites, *Compos. Struct.* 290 (2022) 115462.
- [6] G. Banna, M. Laforest, S. Prudhomme, Generation of pairs of ellipses and ellipsoids from distributions in parameter space, *Comput. Part. Mech.* 12 (2025) 2775–2785.
- [7] P.S. Heckbert, *Graphics Gems IV*, Morgan Kaufmann, 1994.
- [8] M.-A. Chadil, S. Vincent, J.-L. Estivalèzes, Gas-solid heat transfer computation from particle-resolved direct numerical simulations, *Fluids* 7 (2022) 15.
- [9] M. Jbeili, J. Zhang, System-dependent behaviors of nanofluids for heat transfer: a particle-resolved computational study, *Comput. Part. Mech.* 10 (2022) 465–480.
- [10] X. Xing, X. Ku, J. Lin, Z. Yu, Coarse-grained CFD-DEM simulation of biomass particle motion and breakage in a fluidized bed, *Chem. Eng. Sci.* 316 (2025) 121961.
- [11] G.C. Banna, *Estimating Distances Between Ellipsoids with Machine Learning*, (Ph.D. thesis), Polytechnique Montreal, 2023.
- [12] A. Lin, S.-P. Han, On the distance between two ellipsoids, *SIAM J. Optim.* 13 (2002) 298–308.
- [13] E.G. Gilbert, D.W. Johnson, S.S. Keerthi, A fast procedure for computing the distance between complex objects in three-dimensional space, *IEEE J. Robot. Autom.* 4 (1988) 193–203.
- [14] G. van den Bergen, A fast and robust GJK implementation for collision detection of convex objects, *J. Graph. Tools* 4 (1999) 7–25.
- [15] M.E. Abbasov, The method of charged balls for finding the distance between two ellipsoids, *Appl. Math. Comput.* 256 (2015) 118–124.
- [16] G. Tamasyan, The distance between two ellipsoids, *Optimization* 63 (2014) 1845–1858.
- [17] A. Uteshev, M. Yashina, Computation of the distance between two ellipsoids, *Int. J. Solids Struct.* (2008) 1322–1349.
- [18] N.J. Higham, *Accuracy and Stability of Numerical Algorithms*, Society for Industrial and Applied Mathematics, 2002.
- [19] U. Krey, A. Owen, *Basic Theoretical Physics – a Concise Overview*, Springer, 2007.
- [20] S. Chapra, R. Canale, *Numerical Methods for Engineers*, McGraw-Hill, 2009.
- [21] R. Fletcher, *Practical Methods of Optimization*, Wiley, 2000.
- [22] Y. You, Y. Zhao, Discrete element modelling of ellipsoidal particles using super-ellipsoids and multi-spheres: A comparative study, *Powder Technol.* 331 (2018) 179–191.
- [23] K.-A. Sohn, B. Juttler, M.-S. Kim, W. Wang, Computing distances between surfaces using line geometry, in: 10th Pacific Conference on Computer Graphics and Applications, 2002. Proceedings, 2002, pp. 236–245.
- [24] A. Rezghi, J. Zhang, Tank-treading dynamics of red blood cell in shear flow: On the membrane viscosity rheology, *Biophys. J.* 121 (2022) 3393–3410.
- [25] J. Zhang, Convection effect of plasma flow on oxygen transport in capillaries: An in-depth numerical investigation, *Microcirculation* 32 (2025) e70011.
- [26] S.K. Das, M. Cenovic, J. Zhang, A physics-based estimation of mean curvature normal vector for triangulated surfaces, *Proc. Int. Geom. Cent.* 12 (2019) 70–78.



**Dr. Dariusz Amirkhani** is currently a Postdoctoral Fellow at the School of Engineering and Computer Science, Laurentian University, Canada. He received his Ph.D. degree in Information Science and Technology from the Université du Québec en Outaouais (UQO), Canada. His research interests include deep learning, computer vision, image processing, machine learning, and computational modeling of complex fluids.



**Prof. Junfeng Zhang** (Ph.D., P.Eng.) has been working in Mechanical Engineering at Laurentian University (Sudbury, Canada) since 2007. He obtained his Ph.D. degree in Mechanical Engineering from University of Alberta (Edmonton, Canada) in 2005, and then worked as a postdoctoral fellow in Biomedical Engineering at Johns Hopkins University (Baltimore, US). His research focuses on fundamental issues in microscopic complex flows; and interested topics include microscopic blood flows, microfluidics, heat and mass transfer, porous and particulate flows, numerical modeling, machine learning, and lattice Boltzmann method.